

simple-rag-ask-my-notes

A from-scratch RAG tutorial

Architecture, file structure, tools, and the six pipeline phases.

Read this even if you have not opened src/ yet.

Contents

1. 1. What problem RAG solves
2. 2. How this project is organized
3. 3. Tech tools and why each one
4. 4. File structure
5. 5. Architecture / data flow
6. 6. Phase-by-phase lessons
7. 7. Config knobs
8. 8. The sample wiki
9. 9. How to run
10. 10. How to verify RAG is actually happening
11. 11. What v1 does not include

This document is the textbook for the simple-rag-ask-my-notes project. You can read it without opening any Python file. The commented modules in `src/` are a second learning path that shows the same ideas as running code.

1. What problem RAG solves

A large language model is good at fluent sentences. It is not a search engine over your private wiki. If you ask "What is our office Wi-Fi password?", a general chat model will guess, refuse, or invent a plausible password. It has never read your notes.

RAG means Retrieval-Augmented Generation:

1. Retrieve the snippets of your documents that look relevant to the question.
2. Augment the prompt by pasting those snippets in as context.
3. Generate an answer that is allowed to use only that context.

The model still writes the English. The facts are supposed to come from your files. That is the whole trick. Hallucinations still happen if retrieval misses the right snippet, or if the prompt does not forbid guessing. This project makes both of those steps visible.

2. How this project is organized

simple-rag-ask-my-notes is a command-line app, not a website. You ingest a folder of markdown notes once, then you ask questions.

There are two ways to learn it:

- This guide (and the PDF built from it). Architecture, files, tools, and why each phase exists.
- The `src/` package. Each file starts with a docstring that names the RAG phase, why it exists, and what goes wrong if you skip it. Important lines have concept comments (why overlap exists, why top-k matters), not "this imports os".

v1 does not use LangChain, LlamaIndex, or Haystack. Those libraries hide load / chunk / embed / store / retrieve / prompt behind framework objects. After you understand the six phases, rebuilding the same app with LangChain is a useful next step. Doing it first would skip the lesson.

Embeddings and answers run on your machine. There is no OpenAI (or other cloud LLM) API key. ingest uses Chroma's ONNX MiniLM model. ask calls a local Ollama server.

3. Tech tools and why each one

Tool | Role in this project

Python 3.11+ | Language of the tutorial. 3.11 includes tomllib, so the config file needs no extra YAML library.

Chroma ONNX all-MiniLM-L6-v2 | Turns chunk text and the question into vectors in the same space. Runs on disk; first ingest downloads the model once.

Ollama llama3.2:1b | Writes the final answer from the retrieved notes. Temperature defaults to 0 so it copies rather than invents.

openai Python SDK | Talks to Ollama's OpenAI-compatible local API. No wrapper framework.

Chroma | Local vector database on disk at ./chroma_db. Ingest and ask are separate commands; Chroma is the shared memory.

python-dotenv | Loads optional OLLAMA_BASE_URL from .env.

pytest | Unit tests mock chat and embeddings so CI never needs Ollama running.

argparse | Stdlib CLI: ingest and ask.

TOML / config.toml | Knobs: chunk size, top_k, temperature, model names, paths.

What v1 deliberately does not use:

- LangChain / LlamaIndex / Haystack -- would hide the pipeline.
- Docker -- extra moving parts for a folder of Python files.
- Cloud vector DBs -- Chroma on disk is enough and inspectable.
- Cloud LLM APIs -- Ollama + ONNX keep the tutorial runnable with no billing.
- A UI -- stdout is easier to follow with --show-chunks.

4. File structure

```
simple-rag-ask-my-notes/
  config.toml           Knobs (not secrets).
  .env.example          Optional OLLAMA_BASE_URL (default is local Ollama).
  .env                  Overrides for that URL (gitignored). Not an OpenAI key.
  requirements.txt       App libraries.
  requirements-docs.txt  fpdf2, only to rebuild this PDF.
  data/                 Five sample wiki pages (the only corpus in v1).
    onboarding.md
    wifi-and-office.md   Unique fact: password orchid-42.
    project-atlas.md      Atlas launched March 12, 2024; codename Nimbus.
    project-atlas-v2.md   Atlas v2 launched March 15, 2025; migration Horizon.
    team-rituals.md       Irrelevant to Wi-Fi; unique Zoom link.
  src/
    load.py              Phase 1: read .md files.
    chunk.py              Phase 2: overlapping windows.
    embed.py              Phase 3: local ONNX embeddings.
    index.py              Phase 4: Chroma reset + upsert.
    retrieve.py           Phase 5: top-k similarity search.
    generate.py           Phase 6: prompt + Ollama chat completion.
    config.py             Parse and validate config.toml.
    cli.py                Conductor: ingest and ask.
  tests/                 Mocked chat client; real chunking and Chroma.
  chroma_db/             Created by ingest (gitignored).
  docs/
    RAG-GUIDE.md         This file.
    simple-rag-ask-my-notes-guide.pdf
  scripts/build_guide_pdf.py
```

cli.py is the index of the pipeline. The other src/ modules do not call each other in a web of helpers; ingest and ask chain them in a straight line.

5. Architecture / data flow

Two commands, six phases.

Ingest (python -m src.cli ingest):

1. Load markdown from data/.
2. Chunk into overlapping windows; tag each window with filename + index.
3. Embed all chunk texts with local MiniLM (one batched call).
4. Reset the Chroma collection and write vectors + metadata.

Ask (python -m src.cli ask "question"):

5. Embed the question with the same model; retrieve top-k chunks.
6. Build a prompt that forbids guessing; call Ollama llama3.2:1b; print answer + sources.

```
data/*.md --> load --> chunk --> embed --> Chroma (chroma_db)
question --> embed --> retrieve <-- Chroma
retrieve --> prompt + generate --> answer + filenames
config.toml is read by cli.py and passed into those steps.
```

--show-chunks prints the retrieved text before step 6. That is how you tell a retrieval mistake from a generation mistake.

6. Phase-by-phase lessons

Load (src/load.py)

What: Open every *.md file, store filename + full text as a Document.

Why the phase exists: Everything downstream is "about" these files. The filename must be captured now or you cannot cite it later.

If you skip it: There is nothing to chunk or embed. A chatbot with no corpus is just a chatbot.

Config: paths.data_dir

Exercise it: python -m src.cli ingest

Chunk (src/chunk.py)

What: Sliding windows of chunk_size characters, stepping forward by chunk_size - overlap. Each Chunk keeps source and chunk_index.

Why: Embeddings describe a region of text. A whole onboarding page mixes Wi-Fi, Atlas, and Slack. A 500-character window is small enough to be about one thing. Overlap is insurance for facts that sit on the cut.

If you skip it: One vector per file. A Wi-Fi question may lose to the rest of the page. A fact split across a naive cut may match nothing.

Config: chunking.size, chunking.overlap (re-ingest after changes)

Exercise it: python -m src.cli ingest

This project chunks by characters, not tokens, on purpose. Tokenizers hide the idea. Production systems often split on markdown headings or token counts; that is a next step, not v1.

Embed (src/embed.py)

What: Local ONNX all-MiniLM-L6-v2 (bundled with Chroma) returns one vector per string. The first ingest downloads the ONNX archive into ~/.cache/chroma. There is no embeddings.create call to OpenAI.

Why: Similarity search needs a shared geometry. "Wi-Fi password" and a sentence about orchid-42 should land near each other even if the wording differs.

If you skip it: Chroma cannot rank chunks. If you embed notes with model A and the question with model B, nearest neighbor is noise.

Config: embedding.model (re-ingest after changes)

Exercise it: ingest (all chunks) and ask (the single question)

No cloud API key is required here. Tests inject a fake embedder so CI never downloads the ONNX model.

This project does not import or depend on the sentence-transformers package. all-MiniLM-L6-v2 originated as a Sentence Transformers model, but ingest loads Chroma's ONNX wrapper (ONNXMiniLM_L6_V2 in src/embed.py), not SentenceTransformer("all-MiniLM-L6-v2"). Same model family, different runtime. requirements.txt has no sentence-transformers dependency.

Store (src/index.py)

What: Persistent Chroma client. Delete the collection if it exists, create it empty, upsert ids like wifi-and-office.md::0 with document text, embedding, and metadata {source, chunk_index}.

Why: Ask happens later, in another process. Vectors have to live on disk. Metadata is how we rebuild a Chunk without re-reading data/.

If you skip reset: Yesterday's chunks from a deleted paragraph still retrieve. v1 has no incremental update, so rebuild is the honest design.

Config: paths.chroma_path, paths.collection_name (re-ingest after changes)

Exercise it: python -m src.cli ingest

Retrieve (src/retrieve.py)

What: collection.query(query_embeddings=[q], n_results=k) returns the nearest chunk texts.

Why: The generator should see a handful of relevant paragraphs, not the whole wiki and not zero files.

If you skip it: The model answers from training data. That is not RAG.

Config: retrieval.top_k (no re-ingest)

Exercise it: python -m src.cli ask "..." --show-chunks

top_k = 4 is a teaching default. k = 1 misses a fact split across two windows. k = 20 packs the prompt with noise. Edit config.toml and watch --show-chunks change immediately.

Prompt + generate (src/generate.py)

What: A system prompt says: use ONLY the provided notes; if they lack the answer, say you don't know; never invent facts; cite filenames. The user message lists chunks as [filename] blocks plus the question. Then `chat.completions.create` against local Ollama (OpenAI-compatible URL `http://localhost:11434/v1`, model `llama3.2:1b` by default) with temperature from config.

Why: Retrieval is not the answer. Someone still has to write a sentence. The prompt is the contract that keeps that sentence tied to the notes. The SDK is still openai; the server is Ollama on localhost, not `api.openai.com`.

If you skip the contract: The model will happily add a CEO pet name. If you skip the call when retrieval is empty, you never see the don't-know path; v1 still calls the model with empty context on purpose.

Config: `generation.model`, `generation.temperature` (no re-ingest)

Exercise it: `python -m src.cli ask "..."`

Temperature 0 means "copy from notes." Raising it (for example 0.2) can make prose nicer and also more willing to invent.

7. Config knobs

File: `config.toml`. Parsed by `src/config.py`. `cli.py` is the only module that reads Settings; the phase functions take plain arguments so tests do not depend on your edits.

Key | Default | Meaning | Fresh ingest?

`chunking.size` | 500 | Max characters per chunk | yes

`chunking.overlap` | 80 | Characters shared by adjacent windows | yes

`retrieval.top_k` | 4 | How many chunks enter the prompt | no

`embedding.model` | `all-MiniLM-L6-v2` | Must match at ingest and ask | yes

`generation.model` | `llama3.2:1b` | Chat model (Ollama) | no

`generation.temperature` | 0.0 | 0 copies; higher invents more | no

`paths.data_dir` | `data` | Folder of .md notes | yes

`paths.chroma_path` | `chroma_db` | Chroma directory | yes

`paths.collection_name` | `notes` | Collection inside Chroma | yes

Validation: `top_k >= 1`, `chunk_size > overlap >= 0`, temperature in 0.0-2.0, model names non-empty. Failures are `SystemExit` messages, not stack traces.

`--config path/to/config.toml` selects a file. There is no `--top-k` flag in v1; the file is the teaching surface. Cloud API keys are never in this file. Optional `OLLAMA_BASE_URL` belongs in `.env` (default `http://localhost:11434/v1`). `llama3.2:1b` is the default chat model because it fits modest disk and RAM; a larger Ollama tag (for example `llama3.2`) is a config change plus ollama pull, with no ingest.

8. The sample wiki

Five short internal-wiki pages. They overlap on purpose so retrieval has to choose.

- `onboarding.md` -- First week at 14 Harbor Lane. Points you at the Wi-Fi page but does not include the password. Assigns new hires to original Project Atlas. Unique fact: Slack `#arrivals`.

- wifi-and-office.md -- Network Lumen-Office. The password orchid-42 appears only here. Printer code 4419. Atlas war room 4B.

- project-atlas.md -- Billing platform launched March 12, 2024, PostgreSQL, Maya Chen, codename Nimbus.
- project-atlas-v2.md -- Rewrite launched March 15, 2025 (easy to confuse with the other March date), MongoDB, Jordan Okonkwo, migration Horizon.

- team-rituals.md -- Standup, retro, Friday demo. No Wi-Fi, password, or orchid. Unique Zoom zoom.us/j/5550182. Mentions Atlas and Atlas v2 sharing the demo, so product questions might retrieve it; a Wi-Fi question should not rank it first.

This overlap is the lesson. A toy corpus with one file per unrelated topic would make retrieval look easier than it is.

9. How to run

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python -m src.cli ingest
```

ingest needs no cloud key. The first run downloads the MiniLM ONNX model.

For ask, install Ollama (official script needs sudo; otherwise follow the Linux manual install into ~/.local). Then:

```
export PATH="$HOME/.local/bin:$PATH" # skip if ollama is already on PATH
ollama serve                        # leave this running
```

In another terminal:

```
ollama pull llama3.2:1b
python -m src.cli ask "What is the office Wi-Fi password?"
python -m src.cli ask "What is the office Wi-Fi password?" --show-chunks
python -m src.cli --config config.toml ask "When did Project Atlas launch?" --show-chunks
```

If Ollama is not running, ask exits with an install/ollama pull llama3.2:1b hint.

Tests (no Ollama required):

```
pytest -v
```

Rebuild this PDF after editing the markdown:

```
pip install -r requirements-docs.txt
python scripts/build_guide_pdf.py
```

10. How to verify RAG is actually happening

Use a fresh local ingest (no cloud key), with Ollama running, then:

1. Ask for the office Wi-Fi password with --show-chunks. You should see orchid-42 in a wifi-and-office.md chunk before the model answers. That is retrieval, not the model memorizing the question.

2. The printed answer should cite wifi-and-office.md and repeat orchid-42.
 3. team-rituals.md should not be the main hit for that question.
 4. "When did Atlas launch?" should show both March 2024 and March 2025 in the chunks, or the answer should distinguish Atlas vs Atlas v2.
 5. "What is the CEO's pet's name?" must be "I don't know" (or equivalent). The notes have no such fact.
 6. Onboarding may appear for a Wi-Fi question because it mentions Wi-Fi, but it must not be the source of the password string.
 7. Set top_k = 1, ask with --show-chunks -- one chunk. Set top_k = 8 -- more chunks. Do not ingest between those two.
 8. pytest -v stays green with Ollama not running.
 9. With Ollama stopped, ask prints: install Ollama, then ollama pull llama3.2:1b.
- If (1) fails, the bug is chunking, embeddings, or top-k. If (1) succeeds and (2) fails, the bug is the prompt or the chat model.

11. What v1 does not include

Left out on purpose so the six phases stay visible:

- Rerankers (a second model that reorders the top-k)
- Hybrid search (keyword + vectors)
- Chat history / multi-turn
- Agents
- Streaming tokens
- Auth and frontends
- Ingesting PDFs or web pages as notes (this PDF is a guide, not corpus)
- LangChain (rebuild the same pipeline with it after you can explain each file in src/)

Validation of ingest inputs, ask requests, and retrieve/generate outputs is a later product layer. See README Later: validation (after the v1 follow-ups) -- do not add it until the v1 follow-ups above are done.

When you add those later, you will know which phase they sit on top of. That is the point of building this the long way first.